GitHub Copilotで、

業務課題をPython Webアプリにしてみよう

課題整理から、設計、実装、Azureでの実行まで

手作業が多い…

情報がバラバラ…

集計が大変…

あなたの業務課題

GitHub Copilot

設計・構成選択・実装



Azureで動くWebアプリ

個人で開発

困った場合は共同実施OK

初心者歓迎

今日つくるもの

自分の業務課題を、Azureで動くWebアプリにします

体験用ログイン

一覧表示



登録・編集

データ保存

保存先はAIが選ぶ
（Blob Storage／Cosmos DB）

注意：ログインは体験用のデモ機能です。本人確認やアクセス制御を行う認証機能ではありません。

今日持ち帰れるもの

アプリだけでなく、もう一度つくる方法も持ち帰ります

1. **Webアプリ**
   Python / Flask
2. **設計書**
   Markdown
3. **プロンプト**
   00〜10
4. **Azure環境の設計図**
   Terraform
5. **再現手順**
   ローカル / Azure
6. **社内説明資料**
   1ページ概要

「できたアプリ」と「もう一度つくる方法」をセットで持ち帰ります

全体ロードマップ（90分）

必須はSTEP08でPC上、標準はSTEP09でAzure上で動かすこと

考える	設計する	つくる	動かす	持ち帰る
00 01	02 03 04 05	06 07	08 09	10
環境確認・課題整理	要件・画面・データ・Azure構成の選択	Azure構築・プログラム	自分のPC → Azure	再現手順・説明資料

08 必須ゴール（全員）

自分のPCでアプリが動く

09 標準ゴール（全員で挑戦）

AIが選んだ構成をAzureに作り（06）、
Azure上でアプリが動く（09）

遅れても大丈夫です。途中から再開できる完成版をスタッフが用意しています。

準備①

テンプレートをダウンロードして、この資料を開きます

https://github.com/KacomsInstructor/copilot-webapp-handson

1

テンプレートURLを開く

配布されたURLを開き
緑の「Code」
→「Download ZIP」

2

C:¥handson に置く

C:¥handson フォルダーを作り
ダウンロードした
ZIPファイルを置く

3

ZIPを展開

ZIPを右クリック
→「すべて展開」
展開先の欄を C:¥handson にする

4

フォルダー名を変更

末尾の「-main」を消して
copilot-webapp-handson
にする

展開できた状態（エクスプローラー）

```
C:¥handson¥copilot-webapp-handson
├ .github
├ docs
├ guide          ← この資料があります
├ scripts
└ README.md
```

この資料を開く

guide¥handson-guide.pptx

をダブルクリックで開きます。
ここから先は、自分のPCでこの資料を見ながら進めます
。

準備②

必要なソフトがそろっているか確認し、足りないものを入れます

Windowsの「スタート」→「PowerShell」を開き、左の確認コマンドを入力します。OKにならないものだけ、右のコマンドでインストールします。

ソフト	確認コマンド → OKの目安	入っていないとき（インストール）
1 VS Code	`code --version` OK：バージョンが表示	`winget install -e --id Microsoft.VisualStudioCode`
2 Python	`python --version` OK：3.10 以上	`winget install -e --id Python.Python.3.12`
3 Azure CLI	`az version` OK：バージョンが表示	`winget install -e --id Microsoft.AzureCLI`
4 Terraform CLI	`terraform version` OK：v1.5 以上	`winget install -e --id Hashicorp.Terraform`

インストールしたら

PowerShellとVS Codeのウィンドウをすべて閉じて開き直してから、もう一度確認します（開き直さないと「認識されません」と出ます）。
VS Codeの「HashiCorp Terraform」拡張機能だけでは terraform コマンドは使えません。CLI本体が必要です。
「管理者権限が必要」と表示された場合は、手を挙げてスタッフを呼んでください。

準備③

VS Codeでフォルダーを開き、サインインしてセットアップします

VS Codeで開く

1. ファイル → フォルダーを開く
2. C:¥handson¥copilot-webapp-handson を選ぶ
3. 確認画面は「はい、作成者を信頼します」
4. 左側に .github が一番上に見えればOK

GitHubにサインイン

1. 左下の人型アイコン（アカウント）
2. 「GitHubでサインイン」を選ぶ
3. ブラウザーで配布カードのGitHubアカウントを入力
4. 右上のCopilotアイコンでチャットが開く

セットアップを実行

1. 「ターミナル」→「新しいターミナル」
2. 下のコマンドを入力して Enter
3. 「Setup completed.」と表示される
4. 数分かかっても、そのまま待つ

ターミナルに入力

```
.¥scripts¥setup-local.cmd
```

.github が見えない場合は、1つ外側のフォルダーを開いています。copilot-webapp-handson を開き直します。
個人のGitHubアカウントでサインイン済みの場合は、サインアウトして配布カードのアカウントでサインインし直します。
ここでは、まだCopilotにプロンプトを入力しません。

Copilotでは、毎回同じ6つの操作を繰り返します

1 新しいチャットを開く

＋

2 Agentモードを確認

Agent

3 「/」を入力

/

4 プロンプトを選ぶ

/02-requirements

5 送信する

▶

6 生成ファイルを確認

docs/

STEPごとに「＋」で新しいチャットを開きます。前の内容は、作成されたMarkdownファイルから読み込まれます。

STEP 00 環境確認

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

これからの作業を始められる状態か確認します。

ターミナルに入力　`az login`

いまやること

1. 配布カードのAzureアカウントでサインイン
2. チャットのモードを「Agent」にする
3. /00-environment-check を実行する

できあがるもの

- [ ] `docs/00-environment-check.md`

ここまでできたら次へ

- [x] Copilotが応答する
- [x] 結果に「NG：0件」と表示される
- [x] terraform version が表示される（1.5以上）

このSTEPでは…

ここでは、アプリをつくりません。
NGがあれば手を挙げてスタッフを呼んでください。

STEP 01 業務課題の整理

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

アイデアソンの内容を「開発できる課題1つ」に絞ります。

いまやること

1. docs/01-input.md に記入・保存する
2. 「＋」で新しいチャットを開く
3. /01-business-problem を実行する

できあがるもの

- [ ] **docs/01-business-problem.md**

ここまでできたら次へ

- [x] 誰の課題か分かる
- [x] 解決したいことが1つ
- [x] 何人で使うか・管理したい情報がある

このSTEPでは…

何人で使うか、同時に更新する人がいるかも書きます。
STEP05でAIが構成を選ぶ材料になります。

STEP 02 要件定義

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

業務課題を、アプリに必要な機能へ変換します。

いまやること

1. 「＋」で新しいチャットを開く
2. /02-requirements を実行する
3. 機能が3つ以内か確認する

できあがるもの

- [ ] **docs/02-requirements.md**

ここまでできたら次へ

- [x] 機能が F-01〜F-03 まで
- [x] 「今回作らないもの」がある
- [x] 各機能に完成の条件がある

このSTEPでは…

機能を増やす時間ではありません。
最大3つに絞ります。

STEP 03 画面設計

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

利用者が操作する画面と、その流れを決めます。

いまやること

1. /03-screen-design を実行する
2. 画面名・入力項目・ボタンを確認する
3. 各機能に対応する画面があるか確認する

できあがるもの

- [ ] `docs/03-screen-design.md`

ここまでできたら次へ

- [x] 主要画面が3つ以内
- [x] 各機能に対応する画面がある
- [x] デモログインの注記がある

このSTEPでは…

完成アプリの見た目ではなく、
画面の設計図を決めています。

STEP 04 データ設計

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

画面で入力した情報を、保存できる形に整理します。

いまやること

1. /04-data-design を実行する
2. 項目名とデータ例を確認する
3. サンプルデータ3件を確認する

できあがるもの

- [ ] `docs/04-data-design.md`
- [ ] `data/sample-items.json`

ここまでできたら次へ

- [x] 管理するデータが1種類
- [x] 画面の項目と一致している
- [x] サンプルが3件・実在の情報なし

注意

実在する顧客名や個人情報は入力しません。保存先はまだ決めません（次のSTEPでAIが選びます）。

STEP 05 Azure構成の選択

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

課題に合わせて、AIがデータの保存先（Azureの構成）を選びます。

いまやること

1. /05-architecture を実行する
2. docs/05-architecture.md の 1行目でパターンを確認する
3. 選んだ理由を読む

パターンA
Blob Storage

- ✓ 少人数で使う
- ✓ データは数百件程度
- ✓ 一覧と登録が中心

パターンB
Cosmos DB

- ✓ 複数の人が同時に更新
- ✓ 件数が増える
- ✓ 検索・絞り込みしたい

ここまでできたら次へ

- ✓ 「選択：パターンA」または「B」と書かれている
- ✓ 選んだ理由が自分の課題と合っている

このSTEPでは…

隣の人と構成が違っていて大丈夫です。
理由が違うと思えば「同時に更新するのは10人」など伝えて見直してもらいます。

STEP 06 Azure環境をつくる

標準ゴール

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

AIが選んだ構成の設計図（Terraform）をつくり、自分でAzureに構築を始めます。

ターミナルに入力

```
.¥scripts¥init-tfvars.cmd -ResourceGroup <配布カードのRG名>
```

いまやること

1. 上のコマンドを実行する（< > は配布カードの値に置き換える）
2. /06-infrastructure を実行し、plan が成功するまで待つ
3. 「＋」で2つ目のターミナルを開き、名前を apply にする
4. 右下のコマンドを入力して Enter。確認が出たら yes を入力
5. apply のターミナルを閉じずに STEP07 へ進む

できあがるもの

- `infra/terraform/main.tf`
- `infra/terraform/outputs.tf`
- `docs/06-infrastructure.md`

ここまでできたら次へ

- ✓ plan の結果にエラーがない
- ✓ terraform apply を実行し、確認に yes と入力した
- ✓ Azure環境の作成が始まった。apply のターミナルは閉じない

手順3・4のポイント

名前の変更：ターミナル右側の一覧で右クリック →「名前の変更」→ apply

```
terraform -chdir=infra/terraform apply
```

yes はコマンドではありません。確認が出たときだけ入力します。

apply はCopilotに実行させません。

STEP 07 アプリ実装（前半）

いまここ

00 01 02 03 04 05 06 07 08 09 10

● 完了　● いまここ　● これから

設計書をもとに、Copilotが実際のPython Webアプリをつくります。

いまやること

1. /07-implementation を実行する
2. ファイルの変更は「保持」
3. コマンドの確認は内容を見て「許可」

ターミナルの一覧（パネルの右側）

```
▸ powershell    ← テスト・アプリ起動に使う
▸ apply         ← Azure構築中（閉じない）
```

Copilotが進める順番

データの入力チェック（models.py） → 画面の処理（routes.py） → HTML画面（templates/） → テスト（pytest）

ここから初めて、実際のプログラムをつくります。

ターミナルは2つ。右側の一覧で切り替えます。apply のターミナルは閉じずに、ときどき「Apply complete!」を確認します。

アプリ実装（後半）

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

テスト結果に「passed」と表示されれば完了です。

できあがるもの

```
webapp/
├ routes.py
├ models.py
├ templates/
└ static/
tests/test_routes.py
docs/07-implementation-result.md
```

Copilotの作業が終わったら、自分でテストを実行

1. apply とは別のターミナルで、下のコマンドを入力して Enter
2. 結果に passed と表示され、failed が無ければ完了

ターミナルに入力　`.¥.venv¥Scripts¥python -m pytest -q`

表示例：「5 passed in 0.12s」（数字は人によって異なります）

困ったとき

1. 止まって見える → 「許可」ボタンを押す。無ければ「続けて」と送る
2. failed と表示された → エラーを貼り付け「テストを直して」と送る
3. 同じエラーを3回繰り返す／apply のターミナルにエラー → スタッフへ

STEP 08 アプリを動かす

必須ゴール

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

自分のPCでアプリが動けば、今日の必須ゴール達成です。

ターミナルに入力　`.¥scripts¥start-local.cmd`



いまやること

1. apply とは別のターミナルで上を実行
2. http://127.0.0.1:5000 を開く
3. ログインしてデータを登録する
4. Ctrl+Cで止めて、もう一度起動する
5. /08-local-validation を実行する

✓ ブラウザーで開く　✓ ログインできる　✓ 登録して一覧に表示　✓ 再起動してもデータが残る

STEP 09 Azureで動かす

標準ゴール

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

AIが選んだ構成のAzure上で、自分のアプリを動かします。

1. **Azure環境ができたか確認**
   `Apply complete!`
2. **Ctrl+Cで止めてAzureに公開**
   `.¥scripts¥deploy-azure.cmd`
3. **URLを開いてログイン・登録**
   `https://…azurewebsites.net`
4. **保存を確認**
   `/09-azure-validation`

できあがるもの：AIが選んだ構成のAzure環境 ＋ Azure上で動くWebアプリ ＋ docs/09-azure-validation.md

初回の起動や権限の反映に数分かかります。16:07までに動かなくても、STEP08まで完成していれば大丈夫です。

STEP 10 持ち帰り資料

いまここ

00 01 02 03 04 05 06 07 08 09 10

完了　いまここ　これから

帰社後に再現・共有するための資料を、Copilotにつくってもらいます。

「＋」で新しいチャットを開き、 `/10-handover` を実行します。

自社環境で再構築する手順

`docs/10-handover-guide.md`

- [x] ローカルでの実行方法
- [x] 自社Azureでの構築方法

社内説明用の1ページ概要

`docs/11-demo-summary.md`

- [x] 課題と作成したアプリ
- [x] AIが選んだ構成とその理由

ファイルの削除とサインアウトは、成果共有が終わってから全員で行います。

できあがったもの

アプリだけでなく、課題からアプリへ変える工程全体が成果物です



成果物一式：Webアプリ ＋ 設計書 ＋ プロンプト ＋ Azure環境の設計図 ＋ 再現手順

成果共有

ピックアップされた方に、60秒で成果を共有していただきます



1. 選んだ業務課題
2. 作ったWebアプリ
3. AIが選んだAzure構成とその理由
4. どこまで動いたか
5. 帰社後に試したいこと

発表例

「○○という課題を選び、○○できるアプリを作りました。複数人で更新するのでAIはCosmos DBを選びました。」

※ 全員発表ではありません。講師が数名をピックアップします。

持ち帰る前の確認

成果共有が終わったら、認証情報を削除してサインアウトします

成果物の最終チェック

- ✓ プロンプト 00〜10
- ✓ 設計書（docs/）
- ✓ Python Webアプリ
- ✓ Azure環境の設計図（Terraform）
- ✓ 持ち帰りガイド
- ✓ 社内説明用の概要

サインアウトの4ステップ

1. ターミナルで下のコマンドを実行
2. VS CodeでGitHubからサインアウト
3. ターミナルで az logout を実行
4. ブラウザーでGitHub / Azureからサインアウト

1 の入力

```
.¥scripts¥prepare-handover.cmd
```

必ず成果共有が終わってから実行してください。作成したフォルダーは、このPCにそのまま残ります。

全STEPを1枚で確認できます

STEP	いま何をしている？	何ができる？
00	環境確認	開発を始められる
01	課題を1つに絞る	業務課題が整理される
02	必要な機能を決める	要件定義書
03	画面と流れを決める	画面設計書
04	保存する情報を決める	データ設計書
05	AIがAzure構成を選ぶ	構成設計書（パターンA／B）
06	**Azure環境をつくる（標準）**	Terraform・Azureのリソース
07	プログラムをつくる	Python Webアプリ
08	**自分のPCで動かす（必須ゴール）**	アプリが動く
09	**Azureで動かす（標準ゴール）**	Azure上で動く
10	再現資料をつくる	持ち帰り一式

困ったときは、まずこれを試してください

「/」でプロンプトが出ない	.github が一番上に見えるフォルダーを開いているか確認
ファイルが作られない	チャットのモードが「Agent」になっているか確認
同じエラーを繰り返す	停止して、エラー画面を残したままスタッフを呼ぶ
apply がエラーで止まった	エラー画面を残したまま、Azure担当のスタッフを呼ぶ
進行に遅れた	近くの方と一緒に進める（共同実施OK）

一人で止まらず、手を挙げてスタッフを呼んでください。

よくある4つの環境トラブルを切り分けます

terraform が認識されない	Terraform CLI が未導入、またはインストール後に VS Code を開き直していない	**「準備②」の手順で導入し、VS Codeをすべて閉じて開き直す**
az が認識されない	Azure CLI が未導入、またはインストール後に VS Code を開き直していない	**「準備②」の手順で導入し、VS Codeをすべて閉じて開き直す**
Copilot やターミナルが使えない	フォルダーを開いたときに「信頼しない」を選び、制限モードになっている	**画面上部の「制限モード」を押して「信頼する」を選ぶ**
az login 後に自分のRGが見えない	別のアカウント、または別のサブスクリプションが選ばれている	**az account show で確認し、スタッフを呼ぶ。自分でRGを作らない**

解決しない場合は、エラー画面を残したままスタッフを呼んでください。

Copilotの確認画面では、コマンドの種類を見て判断します

「許可」してよい

- ✓ Pythonのテスト（pytest）
- ✓ terraform fmt / init / validate / plan
- ✓ Azureアカウント・Blob・ログの確認

「拒否」してスタッフを呼ぶ

- ✗ Copilotが terraform apply / destroy を実行
- ✗ requirements.txt にないパッケージの追加
- ✗ 権限の変更・ファイルの一括削除

※ terraform apply は、STEP06で受講者自身が新しいターミナルに入力します（Copilotには実行させません）。

今回の体験は、本番化の前段階です

今回の体験会

- ✓ 仮想ログイン（デモ機能）
- ✓ 保存先は Blob か Cosmos DB の2択
- ✓ 主要データ1種類
- ✓ 最大3機能・最大3画面

本番化するとき

- ✓ Microsoft Entra ID などの認証
- ✓ 権限の最小化・閉域ネットワーク
- ✓ バックアップ・監視とログ
- ✓ AIが提案した理想構成の検討（docs/05）

今日つくるのは「本番システム」ではなく、課題をアプリに変える流れを体験するためのものです。

持ち帰ったフォルダーで、別の業務課題にも同じ流れを使えます

1
自分のCopilotを準備
GitHub Copilot

2
setup-local.cmd を実行
環境の再作成

3
01-input.md を書き換え
新しい課題

4
/01 から順に実行
同じプロセス

5
Azureへ構築
管理者準備 → Terraform

プロンプトの再実行には GitHub Copilot の契約が、Azureへの構築には Azure サブスクリプションと、管理者による権限の準備が必要です。